Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

Bài tập & case study **2**
Hướng dẫn sử dụng 2
Chương 1: Từ monolith đến microservices 2
1-0 [NOTE] Ôn tập: Monolith vs Microservices [L1] 2
1-1 [EVAL] Khi nào không nên dùng Microservices? [L2] 3
1-2 [CASE] Case study: Shopify — Monolith tỷ đô [L3] 3
Chương 2: DDD & Bounded Contexts 4
2-0 [NOTE] Ôn tập: các khái niệm Domain-Driven Design [L1] 4
2-1 [DESIGN] Event Storming: thiết kế hệ thống đăng ký tín chỉ [L2] 4
2-2 [EVAL] Shared Library vs API — khi nào nên gì? [L2] 5
Chương 3: Thiết kế API 5
3-0 [NOTE] Ôn tập: các nguyên tắc thiết kế API [L1] 5
3-1 [CASE] Case study: Stripe API — tại sao được coi là “gold standard”? [L2] 6
3-2 [DEBUG] Debugging: Sự cố breaking change của API [L3] 6
Chương 4: Giao tiếp đồng bộ & Resilience 6
4-0 [NOTE] Ôn tập: các resilience pattern [L1] 6
4-1 [DEBUG] Debugging: Cascading Failure — “Tại sao cả hệ thống ngừng hoạt động?” [L3] 7
4-2 [EVAL] REST vs gRPC vs GraphQL — chọn công nghệ nào cho kịch bản tương ứng? [L2] 8
Chương 5: Giao tiếp bất đồng bộ 8
5-0 [NOTE] Ôn tập: các khái niệm Kafka [L1] 8
5-1 [EVAL] Kafka vs RabbitMQ — quyết định dựa trên gì? [L2] 9
5-2 [DEBUG] Debugging: “Thông điệp biến mất” — Dead Letter & Poison Pills [L3] 9
Chương 6: Saga Pattern 10
6-0 [NOTE] Ôn tập: nền tảng Saga [L1] 10
6-1 [DESIGN] Design: Uber Ride Saga — Choreography hay Orchestration? [L3] 10
6-2 [EVAL] Trade-off: Choreography vs Orchestration [L2] 11
Chương 7: Quản lý dữ liệu 12
7-0 [NOTE] Ôn tập: các pattern quản lý dữ liệu [L1] 12
7-1 [EVAL] CAP Theorem trong thực tế — “Pick Two” có đúng không? [L3] 12
7-2 [DESIGN] Design: Chiến lược chuyển đổi database cho KBM [L2] 13
Chương 8: API Gateway 13
8-0 [NOTE] Ôn tập: API Gateway Cross-cutting Concerns [L1] 13
8-1 [CASE] Case study: Netflix Zuul 1, Zuul 2 và Spring Cloud Gateway [L2] 14
8-2 [ADR] ADR: Rate Limiting Strategy [L2] 14
Chương 9: Bảo mật 15
9-0 [NOTE] Ôn tập: Authentication vs Authorization [L1] 15
9-1 [DEBUG] Sự cố bảo mật: đánh cắp JWT token [L3] 15
9-2 [EVAL] OAuth2 + OIDC Flows — chọn flow nào cho kịch bản nào? [L2] 16
Chương 10: Chuyển đổi thực tế 16
10-0 [NOTE] Ôn tập: các chiến lược chuyển đổi [L1] 16
10-1 [CASE] Case study: Amazon — từ monolith, SOA đến microservices (2002–2020) [L3] 17
10-2 [EVAL] Outbox Pattern vs Event Sourcing vs Change Data Capture [L2] 17
10-3 [DESIGN] Design: Đồng bộ dữ liệu và Strangler Fig [L3] 18
Chương 11: Observability 18
11-0 [NOTE] Ôn tập: ba thành phần Observability [L1] 18
11-1 [DEBUG] Debugging: Bí ẩn độ trễ tăng đột biến [L3] 19
11-2 [EVAL] Observability Stack: tự xây, mua hay dùng dịch vụ managed? [L2] 19
Chương 12: Triển khai & DevOps 20
12-0 [NOTE] Ôn tập: các chiến lược triển khai [L1] 20
12-1 [EVAL] Docker Compose vs Kubernetes — ở quy mô nào? [L2] 20
12-2 [DESIGN] Design: CI/CD Pipeline cho Microservices [L3] 21
Capstone: Phỏng vấn thiết kế hệ thống — thiết kế “Online Judge” [L3] 21

Bài tập & case study

> "The best engineers don't just know patterns, they know when NOT to use them."
> — Adapted from system design interview community

Hướng dẫn sử dụng

Mỗi bài tập/case study được phân loại:

Ký hiệu	Loại bài	Mô tả
`DESIGN`	**System Design**	Thiết kế hệ thống từ yêu cầu — dạng phỏng vấn
`EVAL`	**Trade-off Analysis**	Phân tích trade-off giữa các phương án — dạng ByteByteGo
`CASE`	**Case study**	Phân tích hệ thống thực tế (Netflix, Amazon, Stripe…)
`DEBUG`	**Kịch bản gỡ lỗi (Debugging)**	Tìm nguyên nhân gốc rễ từ triệu chứng — dạng xử lý sự cố khi trực on-call
`ADR`	**Architecture Decision Record**	Viết ADR cho một quyết định thiết kế
`NOTE`	**Ôn tập**	Câu hỏi ôn tập kiến thức cốt lõi — kiểm tra hiểu bài

Mức độ: `L1` Cơ bản | `L2` Trung bình | `L3` Nâng cao (mức phỏng vấn)

Chương 1: Từ monolith đến microservices

1-0 `NOTE` Ôn tập: Monolith vs Microservices `L1`

Ghép mỗi đặc điểm (cột trái) với kiến trúc phù hợp nhất (cột phải).

Đặc điểm	Kiến trúc
A. Deploy toàn bộ hệ thống mỗi lần phát hành	1. Microservices
B. Mỗi đội sở hữu và deploy service độc lập	2. Monolith
C. Một database duy nhất cho toàn bộ ứng dụng	3. Modular Monolith
D. Ranh giới module rõ ràng, nhưng deploy cùng nhau	
E. Cần mức trưởng thành DevOps cao (CI/CD, giám sát)	
F. Thích hợp cho đội nhỏ (< 10 người), sản phẩm giai đoạn đầu	

Câu hỏi thêm:

1. Martin Fowler khuyên chiến lược gì khi bắt đầu dự án mới? Giải thích ngắn gọn lý do.
2. Kể tên 3 dấu hiệu cho thấy một monolith *đã sẵn sàng* để tách microservices (tham khảo Bảng 1.1).
3. So sánh cơ chế mở rộng (scaling) giữa kiến trúc Monolith và Microservices. Giải thích tại sao Microservices tối ưu tài nguyên phần cứng hơn khi hệ thống phải xử lý tải trọng không đồng đều giữa các tính năng.

1-1 `EVAL` Khi nào không nên dùng Microservices? `L2`

Tình huống: Người đọc trong vai chuyên gia tư vấn kiến trúc, được ba công ty mời tư vấn "có nên chuyển sang microservices?"

Công ty	Mô tả	Đội ngũ	Lưu lượng	Vấn đề chính
A	Startup fintech, 8 tháng tuổi, chưa rõ product-market fit	4 lập trình viên	200 người dùng/ ngày	Ban giám đốc muốn mở rộng quy mô lớn
B	Thương mại điện tử 5 năm, monolith 500K dòng code, deploy mất 4 giờ	25 lập trình viên, 4 đội	50K đơn hàng/ ngày	Mỗi lần deploy các đội phải phối hợp với nhau, chu kỳ phát hành 2 tuần
C	Hệ thống ERP nội bộ ngân hàng, Java 8, Oracle DB	10 lập trình viên, 2 đội	500 người dùng nội bộ	Muốn "hiện đại hóa" vì đối tác tư vấn đề xuất Microservices

Câu hỏi phân tích:

1. Áp dụng Ma trận quyết định (Bảng 10.1, Chương 10 — đọc trước phần này) cho từng công ty. Chấm điểm cụ thể cho mỗi tiêu chí?
2. Tính **Microservice Premium** cho mỗi trường hợp: chi phí phải trả (độ phức tạp của hệ phân tán, mức trưởng thành DevOps cần có, thời gian học) so với lợi ích thực tế
3. Đưa ra khuyến nghị cho mỗi công ty: **Yes / Not Yet / Never** — kèm lập luận cụ thể
4. Cho công ty B (ứng viên mạnh nhất): service nào nên tách *trước*? Vì sao? Áp dụng nguyên tắc "tách service mang lại nhiều giá trị nhất trước" (Richardson [2a, Ch.13])
5. **Câu hỏi ByteByteGo:** Công ty A muốn người đọc thiết kế microservices "để sẵn cho tương lai". Người đọc sẽ thuyết phục họ thế nào rằng đây là sai lầm? Dùng trích dẫn nào của Martin Fowler?

1-2 `CASE` Case study: Shopify — Monolith tỷ đô `L3`

Bối cảnh: Shopify (2023) vận hành trên **Ruby on Rails monolith** phục vụ hàng triệu nhà bán hàng, xử lý khoảng $236 tỷ GMV. Thay vì chuyển sang microservices, Shopify chọn **Modular Monolith** — tách code thành các component có ranh giới rõ ràng nhưng deploy cùng nhau.

Tham khảo: Shopify Engineering Blog "Deconstructing the Monolith" (2019), "Under Deconstruction" (2020).

Câu hỏi phân tích:

1. Tại sao Shopify — với quy mô lớn hơn hầu hết các công ty — lại *không* chuyển sang microservices? Đây có mâu thuẫn với lời khuyên của Newman không?
2. **Modular Monolith** khác gì **Microservices** về: (a) ranh giới triển khai, (b) cô lập dữ liệu, (c) chi phí giao tiếp (communication overhead), (d) quyền tự chủ của đội ngũ?
3. Shopify dùng khái niệm **Component Boundaries** thay vì **Service Boundaries**. So sánh hai khái niệm này — khi nào component boundary đủ, khi nào cần service boundary?
4. Nếu KBM trong sách áp dụng Modular Monolith thay vì Microservices, kiến trúc sẽ trông thế nào? Vẽ sơ đồ. Trade-off so với kiến trúc hiện tại?

5. **Tranh luận:** "Microservices là bắt buộc cho hệ thống lớn" — người đọc đồng ý hay phản đối? Dùng Shopify làm bằng chứng.

Chương 2: DDD & Bounded Contexts

2-0 `NOTE` Ôn tập: các khái niệm Domain-Driven Design `L1`

Chọn định nghĩa đúng cho mỗi khái niệm DDD:

1. **Bounded Context** là:
   (a) Một cơ sở dữ liệu riêng cho mỗi service
   (b) Ranh giới mà trong đó một mô hình domain có nghĩa nhất quán
   (c) Một design pattern để tách frontend và backend
2. **Ubiquitous Language** là:
   (a) Ngôn ngữ lập trình chung cho toàn dự án
   (b) Từ vựng thống nhất giữa lập trình viên và chuyên gia nghiệp vụ trong cùng một Bounded Context
   (c) Tài liệu API specification bằng OpenAPI
3. **Aggregate** là:
   (a) Một cụm entity được xử lý như một đơn vị nhất quán (consistency boundary)
   (b) Một bảng trong cơ sở dữ liệu quan hệ
   (c) Một microservice chứa nhiều API endpoint
4. Trong KBM, "bài nộp" (Submission) thuộc Bounded Context nào? "Tài khoản sinh viên" thuộc Context nào? Hai Context này giao tiếp qua cơ chế gì?
5. Phân tích vai trò của chuyên gia nghiệp vụ (Domain Expert) trong việc xác định Bounded Context. Tại sao quá trình thiết kế hệ thống Microservices không nên chỉ do đội ngũ kỹ sư phần mềm thực hiện độc lập?

2-1 `DESIGN` Event Storming: thiết kế hệ thống đăng ký tín chỉ `L2`

Tình huống: Người đọc đang thiết kế hệ thống đăng ký tín chỉ (LMS Course Registration) cho học kỳ mới. Các bên liên quan mô tả:

Kịch bản

"Sinh viên đăng nhập hệ thống, tìm kiếm học phần mở trong kỳ. Hệ thống kiểm tra điều kiện tiên quyết. Sinh viên chọn lớp và nhấn đăng ký. Hệ thống kiểm tra sĩ số lớp, nếu còn chỗ sẽ ghi nhận đăng ký thành công và cập nhật sĩ số. Nếu lớp đầy, sinh viên được đưa vào danh sách chờ (waitlist). Khi đăng ký xong, hệ thống gửi thông tin sang Phòng Tài vụ để tính học phí. Sinh viên phải hoàn thành đóng học phí trong 7 ngày, nếu không hệ thống sẽ tự động hủy kết quả đăng ký."

Yêu cầu:

1. **Liệt kê Domain Events** (≥12 sự kiện) theo trình tự: `CourseSearched → PrerequisiteChecked → RegistrationRequested → ...`
2. **Xác định Aggregates:** nhóm các sự kiện → xác định ≥4 aggregate (Course? Registration? Waitlist? Payment? ...)
3. **Vẽ Bounded Contexts:** xác định ≥4 context và quan hệ giữa chúng
4. **Câu hỏi trade-off:** Nên tách "Billing/Payment" thành bounded context riêng hay gộp vào "Registration Management"? Phân tích ưu nhược của cả hai cách

5. **Conway's Law:** Nếu đội chỉ có 8 người, người đọc tổ chức đội thế nào để khớp với các bounded context?

2-2 `EVAL` Shared Library vs API — khi nào nên gì? `L2`

Tình huống: Hai microservice (Order Service và Inventory Service) cần dùng chung logic kiểm tra định dạng mã sản phẩm. Đội ngũ tranh luận:

Phương án	Cách làm	Lập luận ủng hộ
Shared Library	Tạo `common-validation` JAR, cả hai service cùng import	Dễ tái sử dụng, theo nguyên tắc DRY
API Call	Tạo `Validation Service` riêng, gọi qua REST	Liên kết lỏng (loose coupling), deploy độc lập
Duplicate Code	Sao chép logic kiểm tra vào mỗi service	Không coupling, nhưng bảo trì thế nào?

Câu hỏi phân tích:

1. Phân tích trade-off chi tiết cho từng phương án theo 5 tiêu chí: coupling, khả năng deploy độc lập, hiệu năng, bảo trì, quyền tự chủ của đội ngũ
2. KBM sử dụng shared library (`kbm-shared`) chứa DTO, ErrorCode, utils. Đây có phải anti-pattern? Khi nào shared library là hợp lý?
3. Newman trong [4a] khuyên: "Chỉ chia sẻ DTO và cross-cutting concerns (logging, auth). Không chia sẻ business logic." Áp dụng nguyên tắc này: phân loại nội dung `kbm-shared` → cái nào nên giữ, cái nào nên di chuyển về service?
4. **Câu hỏi ByteByteGo:** Người đọc vừa gia nhập một đội mới và thấy một shared library 50 class đang được 8 service sử dụng. Mỗi khi shared library thay đổi, tất cả service phải deploy cùng lúc. Đây là triệu chứng của vấn đề gì? Người đọc đề xuất giải pháp gì?

Chương 3: Thiết kế API

3-0 `NOTE` Ôn tập: các nguyên tắc thiết kế API `L1`

Phân loại mỗi thay đổi API sau đây là **Breaking** hay **Non-breaking**:

Thay đổi	Breaking / Non-breaking?
A. Thêm trường mới `createdAt` vào response JSON	
B. Đổi tên trường `questionTitle` → `title`	
C. Thêm query parameter tùy chọn `?sort=asc`	
D. Thay đổi HTTP status từ `200` → `201` cho POST	
E. Xóa trường `legacyId` khỏi response	
F. Thêm trường bắt buộc `priority` vào request body	

Câu hỏi thêm:

1. Giải thích nguyên tắc **Tolerant Reader** (Postel's Law). Tại sao nguyên tắc này quan trọng cho việc phát triển API theo thời gian (API evolution)?
2. REST API nên dùng chiến lược versioning nào: URL path (`/v2/users`) hay header (`Accept: application/vnd.api+v2`)? Nêu 1 ưu và 1 nhược cho mỗi cách.
3. Đánh giá tầm quan trọng của việc duy trì tính tương thích ngược (backward compatibility) khi thiết kế API trong kiến trúc Microservices so với kiến trúc Monolith. Hậu quả của việc vi phạm nguyên tắc này là gì?

3-1 CASE Case study: Stripe API — tại sao được coi là "gold standard"? L2

Bối cảnh: Stripe API được cộng đồng lập trình viên đánh giá là **một trong những API được thiết kế tốt nhất**. Phân tích tại sao.

Dữ liệu: Truy cập `stripe.com/docs/api` (hoặc dựa trên kiến thức).

Câu hỏi phân tích:

1. Stripe dùng **date-based versioning** (`2024-12-18`) thay vì `v1/v2`. So sánh với URL path versioning — ưu/nhược điểm gì? Khi nào date-based tốt hơn?
2. Stripe API trả response dạng **envelope pattern**: `{ "object": "list", "data": [...], "has_more": true }`. So sánh với cách KBM trả `{ "data": [...], "pagination": {...} }` — cái nào tốt hơn? Tại sao?
3. Stripe sử dụng **Idempotency Key** cho mọi POST request (tránh trừ tiền khách hàng 2 lần). Thiết kế cơ chế tương tự cho `POST /api/submissions` của LMS — sinh viên nộp bài 2 lần do mạng chậm → chỉ chấm 1 lần
4. Stripe luôn trả **error object nhất quán** — mọi lỗi đều nằm dưới khóa `error`, kèm ba trường `type`, `code` và `message` (ví dụ `type: "card_error"`, `code: "expired_card"`). So sánh với định dạng lỗi hiện tại của LMS — khoảng trống ở đâu?
5. **Thiết kế:** Nếu người đọc phải thiết kế API cho tính năng mới: "Giảng viên tạo kỳ thi → sinh viên đăng ký → nộp bài → bảng xếp hạng", viết danh sách endpoint theo phong cách của Stripe

3-2 DEBUG Debugging: Sự cố breaking change của API L3

Tình huống sự cố:

Kịch bản

9:00 AM — Đội ngũ deploy Core Service v2.3: đổi trường `questionTitle` → `title` trong response (cho gọn tên). 9:15 AM — Ứng dụng mobile (v1.2, chưa cập nhật) crash khi tải danh sách câu hỏi. 200 sinh viên đang luyện tập bị ảnh hưởng. 9:30 AM — Đội vận hành phát hiện qua dashboard, rollback Core Service về v2.2. 10:00 AM — Bắt đầu họp rút kinh nghiệm (postmortem).

Câu hỏi phân tích:

1. **Nguyên nhân gốc rễ:** Đây là loại breaking change nào? (đổi tên, xóa, đổi kiểu dữ liệu?) Tại sao đổi tên trường tương đương "xóa cũ + thêm mới"?
2. **Phòng ngừa:** Liệt kê ≥3 cơ chế ngăn sự cố này tái diễn (contract testing, kiểm tra schema, chính sách versioning...)
3. **Thiết kế giải pháp:** Nếu buộc phải đổi tên trường, làm thế nào để không gây breaking change? Mô tả quy trình 3 bước
4. **Consumer-Driven Contract Testing:** Giải thích khái niệm này. Nếu ứng dụng mobile có contract test, sự cố này có được phát hiện trước khi lên production không?
5. **ADR:** Viết Architecture Decision Record cho quyết định: "Từ nay response của API chỉ được THÊM trường, không được XÓA hoặc ĐỔI TÊN trường"

Chương 4: Giao tiếp đồng bộ & Resilience

4-0 NOTE Ôn tập: các resilience pattern L1

Ghép mỗi resilience pattern với mô tả đúng:

Pattern	Mô tả
A. Circuit Breaker	1. Giới hạn số request đồng thời đến một service
B. Retry with Backoff	2. Ngắt kết nối khi service downstream liên tục lỗi, tránh lãng phí tài nguyên
C. Bulkhead	3. Trả về giá trị mặc định khi service chính không phản hồi
D. Timeout	4. Thử lại request thất bại với khoảng cách tăng dần
E. Fallback	5. Đặt giới hạn thời gian chờ để tránh thread bị chặn vô hạn

Câu hỏi thêm:

1. Circuit Breaker có 3 trạng thái. Kể tên và mô tả ngắn gọn điều kiện chuyển đổi giữa chúng.
2. Tại sao timeout ở tầng Gateway phải **lớn hơn** timeout ở tầng service bên trong? Nếu ngược lại sẽ xảy ra vấn đề gì?
3. Một OpenFeign client khai báo `@FeignClient(name = "kbm-academic")` — không host, không port. Hãy mô tả chuỗi bước từ cái tên đó tới một địa chỉ IP thật: Eureka giữ vai trò gì, và chuyện gì xảy ra khi service đích đang chạy ba instance?
4. Nếu Eureka registry sập trong 5 phút nhưng các service vẫn chạy, những lời gọi Feign đang có còn hoạt động không? Vì sao? Điều đó nói lên gì về vị trí của registry trong sơ đồ phụ thuộc?

4-1 `DEBUG` Debugging: Cascading Failure — "Tại sao cả hệ thống ngừng hoạt động?" `L3`

Tình huống sự cố (mô phỏng từ thực tế):

Lịch trình

- 14:00 — Database server của Service Tài vụ (kiểm tra công nợ) gặp sự cố phần cứng, mọi truy vấn đều timeout (30s)
- 14:02 — Service Đăng ký tín chỉ gọi Service Tài vụ → timeout 30s → thread pool cạn kiệt
- 14:05 — Service Cổng thông tin gọi Service Đăng ký tín chỉ → cũng timeout → thread pool cạn kiệt
- 14:08 — API Gateway gọi Cổng thông tin → timeout → Gateway thread pool đầy
- 14:10 — **TOÀN BỘ HỆ THỐNG** sập. Sinh viên không xem được cả lịch học. Health check thất bại.
- 14:25 — Kỹ sư trực on-call khởi động lại tất cả service. Hệ thống phục hồi sau 5 phút.
- **Tổng downtime**: 20 phút (14:10–14:30). Ảnh hưởng: 15.000 người dùng.

Câu hỏi phân tích:

1. Vẽ **sơ đồ diễn biến theo thời gian** (kiểu sequence diagram) cho cascading failure. Service nào lỗi trước? Hiệu ứng domino lan thế nào?
2. **Nguyên nhân gốc rễ và tác nhân kích hoạt:** Lỗi phần cứng database là tác nhân kích hoạt (trigger). Nguyên nhân gốc rễ thực sự là gì? (gợi ý: thiếu pattern nào?)
3. Nếu hệ thống có **Circuit Breaker** (Resilience4j) giữa mỗi service call, lịch trình sẽ thay đổi thế nào? Viết lại lịch trình
4. Nếu có Circuit Breaker *và* **Bulkhead**, kết quả khác gì so với chỉ có Circuit Breaker?
5. Thiết kế **cấu hình resilience** cho hệ thống 4 tầng này: timeout, retry, circuit breaker, bulkhead cho mỗi tầng. Giải thích tại sao timeout tầng ngoài phải > timeout tầng trong
6. **Câu hỏi ByteByteGo:** Netflix xử lý hàng tỷ request mỗi ngày. Thiếu circuit breaker ở *một* service có thể làm sập toàn bộ nền tảng. Netflix giải quyết bằng cách nào? (Gợi ý: Hystrix; khi Hystrix ngừng phát triển, hệ sinh thái Spring chuyển sang Resilience4j)

Gợi ý

Nhớ lại khái niệm Circuit Breaker và Bulkhead ở Chương 4.

4-2 `EVAL` REST vs gRPC vs GraphQL — chọn công nghệ nào cho kịch bản tương ứng? `L2`

Tình huống: Người đọc đang thiết kế giao tiếp giữa các microservice cho hệ thống thương mại điện tử:

Giao tiếp	Đặc điểm
A: Mobile app ↔ BFF (Backend For Frontend)	Băng thông hạn chế, cần truy vấn linh hoạt
B: Order Service ↔ Inventory Service (internal)	Thông lượng cao, độ trễ thấp, schema chặt chẽ
C: Admin Dashboard ↔ Product Service	Cần truy vấn dữ liệu lồng nhau (product + reviews + inventory)
D: Public API cho third-party merchants	Ổn định, có tài liệu, có versioning

Câu hỏi phân tích:

1. Với mỗi kịch bản (A-D), chọn REST / gRPC / GraphQL. Giải thích trade-off
2. Có kịch bản nào mà 2 giao thức phù hợp như nhau? Tiêu chí quyết định cuối cùng (tiebreaker) là gì?
3. Netflix dùng gRPC cho giao tiếp giữa các service, GraphQL cho mobile BFF. Spotify dùng gRPC nội bộ + REST cho API công khai. Phân tích: tại sao các công ty lớn đều hướng gRPC cho nội bộ nhưng giữ REST/GraphQL cho bên ngoài?
4. KBM hiện dùng REST cho LMS core, nhưng Network Lab đã có SOAP/REST/gRPC/JNP và DevOps Lab dùng Go/Chi cho một phần API. Nếu phải chọn giao thức cho một ranh giới nội bộ mới — chọn gì? Phân tích chi phí/lợi ích?

Chương 5: Giao tiếp bất đồng bộ

5-0 `NOTE` Ôn tập: các khái niệm Kafka `L1`

Điền vào chỗ trống bằng thuật ngữ đúng: *Topic, Partition, Consumer Group, Offset, Producer, Broker*.

1. _____ là đơn vị logic để phân loại message (ví dụ: `submissions`, `judge-results`).
2. Mỗi topic được chia thành nhiều _____ để xử lý song song.
3. _____ là vị trí (số thứ tự) của message trong partition — consumer dùng nó để theo dõi đã đọc đến đâu.
4. Nhiều consumer instance cùng thuộc một _____ sẽ chia nhau các partition — mỗi partition chỉ được 1 consumer trong group xử lý.
5. _____ là thành phần gửi message vào topic.
6. _____ là server Kafka lưu trữ và phục vụ message.

Câu hỏi thêm:

1. Giải thích sự khác biệt giữa **at-most-once**, **at-least-once**, và **exactly-once**. Luồng xử lý bài nộp (submission pipeline) của KBM nên dùng loại nào? Tại sao?
2. Khi nào nên tăng số partition của một topic? Có rủi ro gì khi tăng quá nhiều?
3. Kết quả chấm bài cần hiện lên màn hình sinh viên ngay khi có. So sánh ba cách đẩy kết quả tới trình duyệt — **polling**, **Server-Sent Events** và **WebSocket** — theo bốn tiêu chí: độ trễ, tải lên server, chiều giao tiếp, và độ phức tạp khi service chạy nhiều instance. Chọn một phương án cho KBM và nêu điều kiện nào sẽ khiến phương án đó không còn phù hợp.

5-1 `EVAL` Kafka vs RabbitMQ — quyết định dựa trên gì? `L2`

Tình huống: 4 kịch bản khác nhau, mỗi kịch bản cần một message broker:

Kịch bản	Mô tả	Yêu cầu đặc biệt
A: Thông báo học phí	Sinh viên nộp tiền → Gửi email nhắc/xác nhận	Đảm bảo gửi đúng 1 lần, thứ tự không quan trọng
B: Lịch sử giao dịch	Mọi giao dịch đóng/rút học phí = event	Thứ tự đặc biệt quan trọng, lưu giữ (retention) vĩnh viễn
C: Điểm danh quẹt thẻ	Hàng chục ngàn sinh viên quẹt thẻ cổng trường	Thông lượng cực cao, mất vài message vẫn chấp nhận được
D: Hàng đợi xử lý file	Báo cáo đồ án: upload → queue → check đạo văn	Phân phối round-robin, consumer xác nhận (acknowledgment)

Câu hỏi phân tích:

1. Cho mỗi kịch bản: chọn Kafka hay RabbitMQ? Giải thích bằng khác biệt kiến trúc (log-based vs queue-based)
2. Kleppmann trong [7, Ch.11] giải thích: Kafka = "distributed commit log" (chỉ ghi nối, consumer tự quản lý offset). RabbitMQ = "message queue" (broker đẩy message, message bị xóa sau khi ack). Mô hình tư duy (mental model) này giúp ra quyết định thế nào?
3. LinkedIn (nơi tạo ra Kafka) xử lý 7 nghìn tỷ message/ngày trên 7 triệu partition (số liệu 2019). Họ *cần* Kafka. KBM có nhiều loại queue khác nhau: submission pipeline, SQL Judge worker dispatch (`kbm-judge-sql` → `kbm-sandbox-*`), DevOps Lab jobs. Luồng nào thật sự cần Kafka, luồng nào chỉ cần DB queue? Đánh đổi giữa "học đúng công cụ" và "dùng công cụ phù hợp"?
4. **Exactly-once delivery:** Kafka hỗ trợ ngữ nghĩa exactly-once theo giao dịch từ bản 0.11. RabbitMQ không hỗ trợ exactly-once sẵn có. Giải thích tại sao exactly-once khó và tại sao "at-least-once + idempotent consumer" thường đủ
5. KBM chọn Kafka cho luồng xử lý bài nộp. Nếu dùng RabbitMQ hoặc DB queue thay thế, kiến trúc thay đổi thế nào? Tính năng nào bị mất?

5-2 `DEBUG` Debugging: "Thông điệp biến mất" — Dead Letter & Poison Pills `L3`

Tình huống sự cố:

Triệu chứng

Kỳ kiểm tra cuối kỳ — 5 sinh viên báo "nộp bài nhưng không nhận kết quả". Giảng viên kiểm tra DB: bản ghi bài nộp vẫn tồn tại, nhưng không có điểm.

Nhật ký điều tra:

- Log của Core Service: ✓ sự kiện `SubmissionCreated` được publish thành công
- Kafka topic `submissions`: ✓ Message vẫn tồn tại (đã kiểm tra bằng `kafka-console-consumer`)
- Log của Judge Service: 5 message ném `JsonParseException` → consumer crash và khởi động lại → message được tiêu thụ lại → crash lại → **vòng lặp vô hạn**
- Nguyên nhân gốc rễ: 5 bài nộp chứa SQL có ký tự Unicode đặc biệt → giải tuần tự hóa (deserialization) thất bại

Câu hỏi phân tích:

1. Đây là ví dụ về **message "Poison Pill"**. Định nghĩa poison pill. Tại sao nó nguy hiểm hơn một message xử lý lỗi thông thường?

2. Không có DLT (Dead Letter Topic): consumer crash → khởi động lại → tiêu thụ lại chính message đó → crash → lặp vô hạn. **45 message khác** (của các sinh viên không gặp lỗi) cũng bị kẹt phía sau poison pill. Giải thích cơ chế (offset của Kafka consumer không được commit khi xử lý lỗi)
3. Thiết kế giải pháp **3 tầng**:
   - Tầng 1: Retry (bao nhiêu lần? chiến lược backoff?)
   - Tầng 2: Dead Letter Topic (định dạng message? cơ chế cảnh báo?)
   - Tầng 3: Dashboard rà soát thủ công (admin xem các message trong DLT, xử lý lại hoặc loại bỏ)
4. Sau khi triển khai DLT, luồng xử lý mới sẽ như thế nào? Vẽ sơ đồ. 5 message lỗi đi đâu? 45 message còn lại có bị ảnh hưởng?
5. **Phòng ngừa:** Làm sao ngăn poison pill ngay từ đầu? (kiểm tra schema, contract testing ở producer)

Gợi ý

Xem lại khái niệm Dead Letter Topic ở Chương 5 và cấu hình `@RetryableTopic`.

Chương 6: Saga Pattern

6-0 NOTE Ôn tập: nền tảng Saga L1

Sắp xếp các bước sau đây theo đúng thứ tự của một Saga "Sinh viên nộp bài" trong KBM, và xác định hành động bù trừ (compensation action) tương ứng:

Các bước (chưa đúng thứ tự):

- Ghi nhận điểm vào Gradebook
- Judge Service chấm bài (chạy code trong sandbox)
- Core Service tạo bản ghi Submission
- Notification Service gửi email kết quả

Yêu cầu:

1. Sắp xếp lại 4 bước theo thứ tự đúng.
2. Nếu bước "Judge Service chấm bài" thất bại (sandbox timeout), hành động bù trừ cho bước trước đó ("tạo bản ghi Submission") là gì?
3. Choreography Saga và Orchestration Saga khác nhau ở điểm nào? Nêu 1 ưu điểm chính của mỗi loại.
4. **Đúng hay Sai:** "Saga đảm bảo ACID transaction xuyên suốt nhiều service." Giải thích câu trả lời.

6-1 DESIGN Design: Uber Ride Saga — Choreography hay Orchestration? L3

Tình huống: Một chuyến xe Uber trải qua các bước:

```
1. Rider request ride
2. Match driver (tìm tài xế gần nhất)
3. Driver accept
4. Trip starts (GPS tracking bắt đầu)
5. Trip ends (tính khoảng cách)
6. Calculate fare (surge pricing, discounts)
```

```
7. Charge rider's payment method
8. Pay driver (sau commission)
9. Send receipt
```

Nếu bước 7 thất bại (thẻ hết tiền):

```
Compensation: Cancel payment → Notify rider "add payment method" → Hold trip record
(chờ retry)
NHƯNG: KHÔNG thể undo trip (đã chạy rồi!) → Đây là "non-compensatable action"
```

Câu hỏi phân tích:

1. Xác định **compensable**, **pivot**, và **retriable** transaction trong 9 bước. Bước nào là điểm không thể quay lui (point of no return)?
2. Uber (thực tế) dùng **Orchestration** cho ride saga — tại sao? So sánh nếu dùng Choreography: cần bao nhiêu sự kiện? Ai theo dõi trạng thái tổng thể?
3. **Vấn đề isolation:** Khách A đặt xe, tài xế X được ghép → trong lúc tài xế X đang trên đường đến → khách B đặt xe, hệ thống cũng ghép tài xế X. Đây là anomaly gì? (Lost update? Dirty read?) Biện pháp đối phó (countermeasure) nào cho kịch bản này?
4. **Thiết kế timeout:** Tài xế không nhận chuyến trong 30 giây → timeout → tìm tài xế khác. Nhưng do mạng chậm, tài xế nhận chuyến ở giây thứ 31 (message đến trễ). Thiết kế cơ chế xử lý "nhận chuyến muộn" (late acceptance)
5. **Liên hệ KBM:** Hãy thiết kế Saga cho **Quy trình công bố điểm** (Ch.6, "Workflow học vụ cũng là saga"): tính điểm dự kiến → giảng viên xác nhận → công bố bảng điểm → đồng bộ read model → thông báo sinh viên. Bước nào là pivot? Nếu bước đồng bộ read model lỗi sau khi đã công bố, xử lý thế nào? Vẽ sơ đồ.

6-2 `EVAL` Trade-off: Choreography vs Orchestration `L2`

Bài tập: Cho 4 kịch bản saga, phân tích nên dùng Choreography hay Orchestration:

Saga	Các bước	Đặc điểm
A: Đăng ký người dùng	Create account → Send verification email → Assign default role	3 bước, đơn giản, tỷ lệ lỗi thấp
B: Đơn hàng thương mại điện tử	Reserve stock → Process payment → Ship → Update loyalty points	4 bước, thanh toán là then chốt, cần giám sát
C: Yêu cầu bồi thường bảo hiểm	Submit claim → Fraud detection → Adjuster review → Payout → Notify	5 bước, có con người tham gia (human-in-the-loop), quy trình kéo dài nhiều ngày
D: Đặt vé máy bay (nhiều chặng)	Book leg 1 → Book leg 2 → Book hotel → Book car → Payment	5 bước, gọi API bên ngoài, thường gặp lỗi một phần

Câu hỏi phân tích:

1. Cho mỗi saga: Choreography hay Orchestration? Lập luận dựa trên độ phức tạp, số bước, xác suất lỗi, mức độ tham gia của con người
2. Ch.6 dùng ngưỡng "hơn 4 bước hoặc có phân nhánh → cân nhắc orchestration" (theo Rocha [5, §4.4]). Người đọc có đồng ý không? Có ngoại lệ không?
3. **Saga D (Đặt vé máy bay)** đặc biệt khó — các API bên ngoài (hãng bay, khách sạn) có thể timeout hoặc trả kết quả mơ hồ. Thiết kế bù trừ cho tình huống: "Đặt chặng 1 thành công, đặt chặng 2 bị timeout — không biết thành công hay thất bại". Đây là "trạng thái không chắc chắn" (uncertain state) — xử lý thế nào?

4. Netflix dùng **Conductor** (engine orchestration mã nguồn mở). Uber dùng **Cadence/Temporal**. Tại sao các công ty lớn tự xây engine orchestration riêng thay vì dùng choreography?

Chương 7: Quản lý dữ liệu

7-0 NOTE Ôn tập: các pattern quản lý dữ liệu L1

Ghép mỗi pattern/khái niệm với bài toán phù hợp nhất:

Pattern / khái niệm	Bài toán
A. Database per Service	1. Cần lưu vết kiểm toán (audit trail) đầy đủ mọi thay đổi trạng thái (ví dụ: giao dịch ngân hàng)
B. CQRS	2. Tải đọc và tải ghi có yêu cầu rất khác nhau (ví dụ: bảng xếp hạng)
C. Event Sourcing	3. Đảm bảo mỗi service có thể phát triển schema độc lập
D. Shared Database	4. Đội nhỏ, cần đơn giản, chấp nhận coupling giữa các service

Câu hỏi thêm:

1. Giải thích ngắn gọn **CAP Theorem**. Tại sao nói "chọn 2 trong 3" là cách hiểu không chính xác?
2. KBM hiện dùng Shared Database. Nêu 2 vấn đề cụ thể mà kiến trúc này gây ra khi muốn tách microservices.

7-1 EVAL CAP Theorem trong thực tế — "Pick Two" có đúng không? L3

Tình huống: Đội ngũ tranh luận về chiến lược database:

Kịch bản

"CAP theorem nói chỉ chọn được 2 trong 3: Consistency, Availability, Partition Tolerance. Vậy nếu partition xảy ra, ta phải chọn giữa C và A. Nhưng hệ thống của ta cần CẢ HAI..."

Câu hỏi phân tích:

1. Martin Kleppmann phản bác cách hiểu "pick two". Ý của ông: chỉ khi có phân vùng mạng mới phải chọn giữa consistency và availability; không có phân vùng thì có thể có cả hai. Giải thích — khác biệt thực tế so với "pick two" là gì?
2. Phân loại các database theo CAP:

Database	Phân loại	Hành vi khi partition
PostgreSQL (single master)	?	?
Cassandra	?	?
MongoDB (replica set)	?	?
Redis Cluster	?	?

3. KBM dùng PostgreSQL cho nhiều dữ liệu giao dịch. Khi primary DB down một khoảng thời gian, hệ thống có thể không khả dụng cho các luồng ghi. Nếu dùng PostgreSQL với streaming replication, trade-off thay đổi thế nào?
4. **Kịch bản trong trường học:** Giỏ đăng ký môn học dự kiến vs Thanh toán học phí — yêu cầu C/A khác nhau:
   - Giỏ môn học: availability > consistency (OK nếu rớt mạng vẫn lưu tạm, có thể thêm bớt sau)
   - Thanh toán học phí: consistency > availability (không chấp nhận trừ tiền 2 lần)

- Thiết kế: dùng database strategy nào cho từng tính năng?

5. **Câu hỏi ByteByteGo:** Bài báo Dynamo của Amazon (SOSP 2007) nêu yêu cầu cho dịch vụ giỏ hàng: "customers should be able to view and add items to their shopping cart even if disks are failing, network routes are flapping, or data centers are being destroyed by tornados." Vì vậy giỏ hàng dùng kho lưu trữ eventually consistent (Dynamo) và chấp nhận có lúc phải hợp nhất các phiên bản giỏ hàng xung đột. Người đọc có đồng ý với trade-off này không? Giải thích bằng CAP

7-2 `DESIGN` Design: Chiến lược chuyển đổi database cho KBM `L2`

Tình huống: KBM hiện có shared database — Core Service và Academic Service (`kbm-academic`) dùng chung PostgreSQL. Người đọc được giao nhiệm vụ tách database.

Ràng buộc:

- Zero downtime (sinh viên đang dùng hệ thống)
- Đội ngũ 2 lập trình viên
- Thời gian: 3 tháng
- Không được mất dữ liệu

Các bảng:

```
Core domain:     questions, submissions, scores, contests
Academic domain: assignments, assignment_questions, student_assignments
Shared:          users (dùng bởi CẢ HAI)
Cross-reference: assignment_questions.question_id → questions.id
```

Câu hỏi phân tích:

1. Áp dụng 5 chiến lược tách database (§7.2, Hình 7.5): thiết kế lộ trình 3 tháng — tháng 1, 2, 3 làm gì?
2. Bảng `users` được cả hai service sử dụng. 3 phương án: (a) giữ ở Core, Academic gọi API, (b) nhân bản dữ liệu users, (c) tạo User Service riêng. Phân tích trade-off của từng phương án
3. `assignment_questions` tham chiếu `questions.id` — đây là một phép join xuyên miền (cross-domain join). Sau khi tách DB, join này không còn hoạt động. Thiết kế giải pháp: API composition? Data duplication? CQRS?
4. **Rủi ro ghi kép (dual write):** Trong quá trình chuyển đổi, có một giai đoạn code cũ (truy cập DB trực tiếp) và code mới (qua API) cùng tồn tại. Làm sao tránh dữ liệu không nhất quán? (Feature flag? Shadow write?)
5. **Kế hoạch rollback:** Nếu tháng 2 phát hiện vấn đề hiệu năng nghiêm trọng — làm cách nào để rollback về shared database an toàn?

Chương 8: API Gateway

8-0 `NOTE` Ôn tập: API Gateway Cross-cutting Concerns `L1`

Từ danh sách sau, đánh dấu những chức năng nào thuộc trách nhiệm của API Gateway (có thể chọn nhiều):

1. ☐ Xác minh JWT ở biên (edge token validation / auth)
2. ☐ Business logic xử lý đơn hàng
3. ☐ Rate Limiting (giới hạn số request/giây)
4. ☐ Request routing đến đúng downstream service

5. ☐ Database migration
6. ☐ SSL/TLS termination
7. ☐ Load balancing
8. ☐ Response caching
9. ☐ Tính toán điểm bài tập (grading logic)

Câu hỏi thêm:

1. Giải thích sự khác biệt giữa **Gateway Routing** và **Gateway Offloading**. Cho ví dụ mỗi loại.
2. Tại sao không nên đặt business logic vào API Gateway? Hậu quả nếu vi phạm là gì?

8-1 `CASE` Case study: Netflix Zuul 1, Zuul 2 và Spring Cloud Gateway `L2`

Bối cảnh: Netflix viết lại API Gateway của mình từ blocking sang non-blocking; hệ sinh thái Spring đi một hướng riêng:

- **Zuul 1**: (mã nguồn mở năm 2013): Blocking I/O, dựa trên servlet, mỗi request chiếm một thread
- **Zuul 2**: (Netflix giới thiệu năm 2016, mã nguồn mở năm 2018): Non-blocking, dựa trên Netty. Năm 2018, Netflix vận hành hơn 80 cluster Zuul 2, xử lý hơn 1 triệu request/giây, làm cửa ngõ cho mọi request vào hạ tầng cloud của Netflix
- **Spring Cloud Gateway**: (bản chính thức đầu tiên năm 2018, trong Spring Cloud Finchley): Dự án của Spring, không phải của Netflix; reactive trên Spring WebFlux và Reactor Netty; được Spring giới thiệu là gateway thế hệ mới thay thế Spring Cloud Netflix Zuul

Câu hỏi phân tích:

1. Chuyển từ Zuul 1 sang Zuul 2 nghĩa là viết lại gateway theo mô hình bất đồng bộ. Cái giá của việc này là gì (độ phức tạp khi lập trình, khi gỡ lỗi, các thư viện đi kèm cũng phải bất đồng bộ)? Vì sao một hệ thống ở quy mô Netflix vẫn chấp nhận trả cái giá đó? (Gợi ý: đọc bài "Zuul 2: The Netflix Journey to Asynchronous, Non-Blocking Systems", Netflix TechBlog, 2016)
2. **Blocking vs Non-blocking Gateway:** Zuul 1 dùng 1 thread/request (blocking). Spring Cloud Gateway dùng event loop (non-blocking). Với 10K kết nối đồng thời, Zuul 1 cần bao nhiêu thread? Spring Cloud Gateway cần bao nhiêu? Hệ quả với bộ nhớ?
3. KBM dùng Spring Cloud Gateway cho LMS core và Go hostname-based router cho DevOps Lab. Với lưu lượng học thuật vừa phải, blocking hay non-blocking có khác biệt? Khi nào KBM *cần* non-blocking gateway/router?
4. **Gateway là điểm lỗi duy nhất (single point of failure):** Nếu Gateway sập, toàn bộ hệ thống ngừng hoạt động. Thiết kế tính sẵn sàng cao cho Gateway thế nào? (Nhiều instance? Load balancer phía trước?)
5. **BFF Pattern (Backend for Frontend):** Netflix có gateway riêng cho ứng dụng mobile (băng thông ít) và ứng dụng web (cần nhiều dữ liệu hơn). Nếu KBM cần hỗ trợ ứng dụng mobile, người đọc sẽ thiết kế BFF thế nào?

8-2 `ADR` ADR: Rate Limiting Strategy `L2`

Tình huống: Hệ thống LMS bị lạm dụng:

- Một sinh viên viết script tự động nộp bài 1000 lần/phút (dò đáp án SQL kiểu brute-force)
- Judge Service quá tải → ảnh hưởng kỳ thi của 200 sinh viên khác

Yêu cầu: Viết Architecture Decision Record cho Rate Limiting.

Mẫu ADR:

1. **Title:** Rate Limiting Strategy cho LMS API Gateway

2. **Context:** Mô tả vấn đề (lạm dụng, tác động)
3. **Decision Drivers:** Bảo mật, Sử dụng công bằng, Hiệu năng, Chi phí triển khai
4. **Options Considered:**
   - Option 1: Rate limit theo người dùng (5 lần nộp/phút) — cấu hình như Listing 8.4
   - Option 2: Rate limit theo IP
   - Option 3: Rate limit theo route + cửa sổ thời gian (endpoint thi nghiêm ngặt hơn)
   - Option 4: Rate limiting thích ứng (tự điều chỉnh theo tổng tải)
5. **Decision:** Chọn option nào? Tại sao?
6. **Consequences:** Tích cực, Tiêu cực, Rủi ro
7. **Follow-up:** Chiến lược giám sát? Cảnh báo khi người dùng bị rate limit?

Chương 9: Bảo mật

9-0 `NOTE` Ôn tập: Authentication vs Authorization `L1`

Phân loại mỗi tình huống sau là **Authentication** (AuthN) hay **Authorization** (AuthZ):

Tình huống	AuthN / AuthZ?
A. Sinh viên đăng nhập bằng username/password	
B. Hệ thống kiểm tra sinh viên có quyền xem bài tập của lớp này không	
C. Gateway xác minh JWT token chưa hết hạn	
D. Service kiểm tra role "ADMIN" trước khi cho phép xóa người dùng	
E. OAuth2 server cấp access token sau khi người dùng đồng ý cấp quyền (consent)	
F. API kiểm tra `scope` trong token có bao gồm `submissions:write` không	

Câu hỏi thêm:

1. JWT gồm 3 phần. Kể tên và mô tả ngắn gọn nội dung mỗi phần.
2. Tại sao sách khuyến nghị dùng RS256 thay vì HS256 cho microservices? Nêu 1 lý do chính.

9-1 `DEBUG` Sự cố bảo mật: đánh cắp JWT token `L3`

Tình huống sự cố:

Báo cáo

Báo cáo: Sinh viên A phát hiện điểm bài tập thay đổi — bài chưa nộp bỗng có điểm 100. Log cho thấy các bài nộp được tạo bằng JWT token của sinh viên A, nhưng từ một địa chỉ IP khác.

Điều tra: Sinh viên B (cùng phòng) sao chép JWT token từ DevTools trên trình duyệt của A khi A quên đăng xuất trên máy dùng chung. Token dùng HS256, thời hạn 24 giờ, không có cơ chế làm mới token (refresh).

Câu hỏi phân tích:

1. **Tại sao JWT stateless lại nguy hiểm trong tình huống này?** Token bị đánh cắp nhưng server không biết — không có cơ chế thu hồi (revoke). So sánh với xác thực dựa trên session (session ID có thể thu hồi ngay lập tức)
2. **Các tầng giảm thiểu rủi ro** — thiết kế 4 tầng bảo vệ:
   - Tầng 1: Thời hạn sống của token (TTL): rút ngắn thời hạn → bao nhiêu phút là hợp lý?
   - Tầng 2: Làm mới và xoay vòng token (refresh + rotation) (Ch.9, Listing 9.1) — giải thích tại sao xoay vòng giúp phát hiện token bị đánh cắp

- Tầng 3: Nhận dạng thiết bị (device fingerprinting): gắn token với thiết bị/trình duyệt
- Tầng 4: Phát hiện bất thường (anomaly detection): nộp bài từ 2 IP khác nhau trong 5 phút → gắn cờ

3. **HS256 vs RS256:** (a) Sinh viên B đánh cắp được một JWT bearer token. B có thể dùng token này để truy cập hệ thống không? B có thể tự tạo token giả với vai trò `ADMIN` không? Trả lời cho cả hai trường hợp HS256 và RS256. (b) Nâng cao: Khác biệt nghiêm trọng hơn xuất hiện khi một service phía sau (downstream) bị xâm nhập — nếu service giữ khóa bí mật dùng chung (shared secret) của HS256 thì kẻ tấn công có thể tự ký token bất kỳ; nếu service chỉ giữ public key của RS256 thì sao? Phân tích phạm vi thiệt hại (blast radius) của từng trường hợp.
4. **Danh sách đen token (token blacklist):** Triển khai danh sách đen (Redis set) để thu hồi các token bị đánh cắp. Trade-off: mất ưu điểm "stateless" của JWT. Khi nào danh sách đen đáng để đánh đổi?
5. **Zero Trust và tin cậy phân tầng (tiered trust):** Tại sao mạng nội bộ (private network) không phải là bằng chứng danh tính? Đối chiếu rủi ro đang được chấp nhận (accepted risk) của KBM hiện nay (tin vào header thuần — plain-header trust — và tự viết mã phân tích JWT) với mục tiêu Tier 1 (signed proof đúng audience + local authorization). Đề xuất thứ tự chuyển đổi không làm gián đoạn các request đang đi qua Gateway.

9-2 `EVAL` OAuth2 + OIDC Flows — chọn flow nào cho kịch bản nào? `L2`

Bài tập: 4 ứng dụng cần tích hợp với Identity Provider (Keycloak/Auth0):

Ứng dụng	Loại	Tương tác người dùng
A: Web app cho sinh viên (React SPA)	Ứng dụng trang đơn, chạy trên trình duyệt	Người dùng đăng nhập, phiên dài
B: Công cụ CLI cho quản trị viên	Không có trình duyệt, chạy trên terminal	Quản trị viên đăng nhập
C: Judge Service (server-to-server)	Không có người dùng, xử lý ở backend	Máy với máy (machine-to-machine)
D: Ứng dụng di động (React Native)	Ứng dụng native, màn hình đăng nhập riêng	Người dùng đăng nhập, sinh trắc học

Câu hỏi phân tích:

1. Với mỗi ứng dụng (A-D): chọn OAuth2 flow nào? (Authorization Code + PKCE, Client Credentials, Device Authorization Grant)
2. **App A (SPA):** Tại sao Authorization Code flow cần PKCE cho SPA? Không có PKCE → kiểu tấn công nào có thể xảy ra?
3. **App C (server-to-server):** Client Credentials flow chỉ định danh workload, không tạo subject là người dùng. Nếu Judge Service cần xử lý thay mặt sinh viên, hãy thiết kế cơ chế delegation có chữ ký, tách subject khỏi actor, giới hạn audience/thời hạn, và nêu cách service đích phân quyền cục bộ (local authorization).
4. KBM đổi danh tính từ nhà cung cấp định danh bên ngoài/QLĐT/tên đăng nhập-mật khẩu lấy KBM JWT (gần với token exchange, Ch.9). Nếu chuyển sang Keycloak hoặc Microsoft Entra ID làm IdP trung tâm: những gì thay đổi? Lợi ích? Chi phí chuyển đổi?

Chương 10: Chuyển đổi thực tế

10-0 `NOTE` Ôn tập: các chiến lược chuyển đổi `L1`

Sắp xếp các bước sau đây của **Strangler Fig Pattern** theo thứ tự đúng:

Các bước (chưa đúng thứ tự):

- (A) Chuyển hướng traffic từ tính năng cũ sang service mới
- (B) Xác định tính năng ít rủi ro nhất để tách trước
- (C) Xây dựng service mới hiện thực tính năng đã chọn
- (D) Gỡ bỏ code cũ khỏi monolith sau khi service mới ổn định
- (E) Chạy song song (canary/shadow) để so sánh kết quả cũ vs mới

Câu hỏi thêm:

1. Nêu 2 anti-pattern phổ biến khi chuyển đổi từ monolith sang microservices (tham khảo Phụ lục D).
2. **Đúng hay Sai:** "Nên tách tất cả service cùng lúc (Big Bang) để tiết kiệm thời gian." Giải thích.
3. KBM nên tách service nào trước? Tại sao? (Gợi ý: xem xét độ độc lập và giá trị nghiệp vụ)

10-1 `CASE` Case study: Amazon — từ monolith, SOA đến microservices (2002–2020) `L3`

Bối cảnh: Chỉ thị nổi tiếng được cho là của Jeff Bezos, ban hành khoảng năm 2002 (theo Steve Yegge kể lại trong "Stevey's Google Platforms Rant", 2011; trích một phần):

Trích dẫn

> "All teams will henceforth expose their data and functionality through service interfaces. ... There will be no other form of interprocess communication allowed ... Anyone who doesn't do this will be fired."

Câu hỏi phân tích:

1. Chỉ thị năm 2002 ra đời *trước* khi thuật ngữ "microservices" phổ biến (bài viết của Lewis và Fowler, 2014). Amazon gọi kiến trúc của họ là "Service-Oriented Architecture". Sự khác biệt giữa SOA của Amazon và microservices hiện đại là gì?
2. Amazon không làm "big bang" rewrite. Họ tách service dần dần qua nhiều năm. Có thể dùng Strangler Fig Pattern (Ch.10) để mô tả quá trình này không? Vẽ lịch trình 3 giai đoạn
3. **Quy tắc "Two-Pizza Team"** — mỗi team 6-8 người, sở hữu 1-2 service. Đây có phải là Conway's Law trong thực tế? Liên hệ với Ch.2 (cấu trúc team ↔ ranh giới service)
4. Amazon hiện có **hàng nghìn** microservices. Vấn đề mới: số service phình to mất kiểm soát (service sprawl), quản lý phụ thuộc, mơ hồ về quyền sở hữu. Giải pháp? (Service mesh, service catalog, platform team)
5. **So sánh với KBM:** KBM có nhiều service/module cho một team nhỏ. Áp dụng quy tắc Two-Pizza: bao nhiêu service là hợp lý cho quy mô team này? KBM có rủi ro "quá nhiều service so với một team nhỏ" không?

10-2 `EVAL` Outbox Pattern vs Event Sourcing vs Change Data Capture `L2`

Tình huống: Team cần giải quyết vấn đề ghi kép (dual-write problem): vừa lưu DB vừa phát event. Có 3 cách tiếp cận:

Cách tiếp cận	Cơ chế	Ví dụ công cụ
Outbox Pattern	Ghi event vào một bảng trong DB (cùng transaction) → polling/CDC phát event	Custom code + cron/Debezium
Event Sourcing	Sự kiện (event) LÀ nguồn sự thật (source of truth), trạng thái được suy ra từ chuỗi sự kiện	Axon Framework, EventStoreDB
CDC (Change Data Capture)	Đọc transaction log của DB → tự động phát các thay đổi	Debezium + Kafka Connect

Câu hỏi phân tích:

1. So sánh 3 cách tiếp cận theo 6 tiêu chí: độ phức tạp, chi phí hạ tầng, độ trễ, độ tin cậy, thời gian học, tác động lên mô hình dữ liệu
2. **Event Sourcing** thay đổi tận gốc mô hình dữ liệu (event là gốc, trạng thái được suy ra). Khi nào trade-off này đáng? Khi nào quá mức? Cho ví dụ domain phù hợp (ngân hàng) và không phù hợp (trang quản trị CRUD)
3. **CDC chạy bên ngoài mã ứng dụng** — không cần thay đổi code. Ưu điểm rõ ràng. Nhưng nhược điểm là gì? (phụ thuộc loại DB, phức tạp khi vận hành, nhạy cảm với thay đổi schema)
4. Ch.10 đề xuất cho KBM Outbox Pattern với polling publisher. Có hợp lý không? Nếu traffic tăng 100 lần, cách tiếp cận nào phù hợp hơn?
5. **Câu hỏi ByteByteGo:** Người đọc đang phỏng vấn. Người phỏng vấn hỏi: "Giải thích vấn đề ghi kép (dual-write) và 3 cách giải quyết, mỗi cách kèm một ví dụ thực tế." — Trả lời trong 3 phút

10-3 `DESIGN` Design: Đồng bộ dữ liệu và Strangler Fig `L3`

Tình huống: Người đọc đang áp dụng Strangler Fig Pattern để bóc tách tính năng "Điểm danh sinh viên" từ hệ thống Monolith cũ sang một Microservice mới. Tuy nhiên, tính năng "Tính điểm chuyên cần" vẫn nằm bên trong Monolith và cần dữ liệu điểm danh.

Câu hỏi phân tích:

1. Nếu Service Điểm danh ghi dữ liệu vào Database mới, làm thế nào Monolith lấy được dữ liệu này để tính điểm?
2. Phân tích trade-off của 3 phương án đồng bộ:
   - (A) Trigger ở Database mới đồng bộ ngược về DB Monolith.
   - (B) Tạo API trên Service mới để Monolith gọi đồng bộ (Sync API).
   - (C) Sử dụng Event Queue (Kafka/RabbitMQ) để Service mới gửi event `AttendanceRecorded` và Monolith hoạt động như Consumer lắng nghe.
3. Kịch bản: Quá trình chuyển đổi kéo dài 6 tháng. Giữa chừng, trường thay đổi logic quy định thời gian điểm danh trễ. Việc duy trì logic này ở cả code Monolith lẫn Microservice tạo ra nguy cơ gì? Giải quyết thế nào?

Chương 11: Observability

11-0 `NOTE` Ôn tập: ba thành phần Observability `L1`

Ghép mỗi câu hỏi vận hành với thành phần Observability phù hợp nhất: **Logs**, **Metrics**, hoặc **Traces**.

Câu hỏi vận hành	Logs / Metrics / Traces?
A. "Mức dùng CPU trung bình của Judge Service trong 1 giờ qua là bao nhiêu?"	
B. "Tại sao request nộp bài của sinh viên X mất 5 giây?"	
C. "Lỗi gì xảy ra lúc 3 giờ sáng khiến service bị crash?"	
D. "Tỷ lệ request thành công (success rate) là bao nhiêu %?"	
E. "Request đi qua những service nào và mỗi service mất bao lâu?"	
F. "Nội dung stack trace của ngoại lệ khi chấm bài thất bại là gì?"	

Câu hỏi thêm:

1. Phân biệt **SLI**, **SLO**, và **SLA**. Cho ví dụ mỗi loại trong bối cảnh KBM.

2. Tại sao cảnh báo nên dựa trên SLO (trải nghiệm người dùng) thay vì metric kỹ thuật (CPU, RAM)?

11-1 DEBUG Debugging: Bí ẩn độ trễ tăng đột biến L3

Tình huống sự cố:

Cảnh báo: Dashboard Grafana báo độ trễ P99 của việc nộp bài tăng từ 2s → 12s. Xảy ra mỗi thứ Ba lúc 14:00.

Dữ liệu hiện có:

- Metrics: CPU của tất cả service < 30%. Bộ nhớ bình thường. Kafka consumer lag tăng lúc 14:00.
- Logs: Không có lỗi. Judge Service log: "Executing SQL..." — bình thường.
- Tracing: Một trace mẫu cho thấy: Gateway (5ms) → Core (20ms) → Kafka publish (10ms) → Judge receive (delay 8000ms!) → Judge execute (800ms) → Core result (15ms)

Manh mối: Thứ Ba 14:00 = đầu giờ "Cơ sở dữ liệu" — 120 sinh viên truy cập hệ thống cùng lúc.

Câu hỏi phân tích:

1. Từ dữ liệu trace, **điểm nghẽn ở đâu?** Kafka consumer delay 8000ms — nguyên nhân có thể là gì? (Gợi ý: consumer lag, số partition, số consumer thread)
2. **RED analysis:** tính Rate, Error rate, Duration tại thời điểm xảy ra sự cố. Metric nào bất thường?
3. Judge Service xử lý từng message tuần tự (chỉ một consumer thread). 120 bài nộp cùng lúc → dồn ứ trong hàng đợi. Thiết kế **chiến lược mở rộng**: (a) tăng số consumer thread/instance, (b) tăng số partition, (c) cả hai? Trade-off?
4. Thiết kế **SLO** cho độ trễ nộp bài. Hiện P99 bình thường là 2s, lúc tăng đột biến là 12s. SLO hợp lý: "99% bài nộp < Xs" — X bao nhiêu? Ngưỡng cảnh báo?
5. Nếu người đọc là kỹ sư trực (on-call) nhận cảnh báo lúc 14:05: mô tả **runbook** — 5 bước đầu tiên cần kiểm tra (từ tổng thể → chi tiết)?

11-2 EVAL Observability Stack: tự xây, mua hay dùng dịch vụ managed? L2

Tình huống: CTO yêu cầu người đọc đề xuất observability stack cho hệ thống 15 microservices:

Phương án	Thành phần	Chi phí (ước tính/tháng)	Công sức thiết lập
A: Self-hosted OSS	Prometheus + Grafana + Loki + Jaeger	Chi phí hạ tầng tự quản	2-3 tuần
B: Managed OSS	Grafana Cloud (free tier → paid)	Theo mức sử dụng	2-3 ngày
C: Commercial	Datadog / New Relic / Dynatrace	Cao hơn, đổi lại có hỗ trợ	1 ngày (cài agent)
D: Cloud-native	AWS CloudWatch + X-Ray / GCP Cloud Monitoring	Theo cloud provider	1-2 ngày

Câu hỏi phân tích:

1. Với 3 hồ sơ team sau, phương án nào phù hợp nhất?
   - Startup 5 người, ngân sách $0-100/tháng
   - Scale-up 20 người, ngân sách $500/tháng, SLA 99.9%
   - Enterprise 100 người, ngành chịu quản lý chặt (ngân hàng)

2. KBM hiện đã có một số mảnh ở Cấp 2 (Actuator/Prometheus/Grafana, zerolog/PLG ở DevOps Lab) nhưng chưa chuẩn hóa đầu cuối. Lộ trình chuyển đổi đề xuất lên Cấp 3: chọn phương án nào? Tại sao?
3. **Vendor lock-in:** Ngôn ngữ truy vấn độc quyền của Datadog so với PromQL của Prometheus (chuẩn mở). Khi nào vendor lock-in chấp nhận được? Khi nào nguy hiểm?
4. OpenTelemetry (OTel) đang trở thành chuẩn chung — "instrument once, send to any backend". Chiến lược: dùng OTel ngay từ đầu → dễ đổi backend về sau. Trade-off so với dùng SDK của nhà cung cấp?

Chương 12: Triển khai & DevOps

12-0 `NOTE` Ôn tập: các chiến lược triển khai `L1`

Ghép mỗi chiến lược triển khai với đặc điểm chính của nó:

Chiến lược	Đặc điểm
A. Blue-Green Deployment	1. Triển khai cho một phần nhỏ người dùng, tăng dần nếu ổn
B. Canary Deployment	2. Duy trì 2 môi trường giống hệt nhau, chuyển traffic tức thì
C. Rolling Update	3. Gửi bản sao traffic đến phiên bản mới nhưng không ảnh hưởng người dùng
D. Shadow Deployment	4. Thay thế từng instance cũ bằng instance mới, từng cái một

Câu hỏi thêm:

1. Docker container khác gì virtual machine? Nêu 2 điểm khác biệt chính.
2. Giải thích nguyên tắc **Infrastructure as Code**. Tại sao không nên cấu hình server thủ công?
3. Với KBM (team nhỏ, Docker Compose), chiến lược triển khai nào phù hợp nhất để bắt đầu? Tại sao?

12-1 `EVAL` Docker Compose vs Kubernetes — ở quy mô nào? `L2`

Tình huống: 3 hệ thống ở các quy mô khác nhau:

Hệ thống	Số service	Traffic	Team	Yêu cầu
A: Startup MVP	3 service	500 req/ngày	2 lập trình viên	Ra mắt nhanh, ngân sách thấp
B: KBM LMS + labs	Nhiều service/ module	Traffic học thuật, tăng đột biến khi thi/lab	1–2 lập trình viên chính (+ sinh viên theo đợt)	Tin cậy, dễ vận hành
C: E-commerce platform	25 service	1M req/ngày	15 lập trình viên, 4 team	Auto-scaling, zero-downtime, nhiều region

Câu hỏi phân tích:

1. Với mỗi hệ thống: Docker Compose hay Kubernetes? Hay kết hợp cả hai? Giải thích dựa trên các tiêu chí: năng lực team, chi phí vận hành, nhu cầu mở rộng
2. **Anti-pattern:** Hệ thống A (3 service, 2 lập trình viên) muốn dùng Kubernetes "để sẵn cho tương lai". Tính **chi phí của Kubernetes** cho team 2 người: thời gian học, bảo trì cluster, độ phức tạp khi debug. Chi phí > lợi ích?
3. **Lộ trình chuyển đổi:** Hệ thống B (KBM) hiện dùng Docker Compose cho LMS core và k3s/ Sysbox cho DevOps Lab. Khi nào *cần* mở rộng Kubernetes cho nhiều phần hơn? Xác định 3 tín hiệu kích hoạt (traffic, quy mô team, yêu cầu SLA)
4. **Managed vs Self-managed K8s:** GKE/EKS/AKS so với tự dựng cluster bằng kubeadm. So sánh chi phí cho hệ thống C? So sánh rủi ro?

5. **Câu hỏi ByteByteGo:** "How does Kubernetes ensure zero-downtime deployment?" — Trả lời: cơ chế Rolling Update, readiness probe, PDB (Pod Disruption Budget). Vẽ sơ đồ

12-2 `DESIGN` Design: CI/CD Pipeline cho Microservices `L3`

Tình huống: Người đọc cần thiết kế CI/CD pipeline cho KBM (Java LMS core nhiều repo/service + DevOps Lab Go multi-binary):

Yêu cầu:

- Push code → tự động build → test → deploy lên staging → phê duyệt thủ công → deploy lên production
- Chỉ build/test các service bị thay đổi (không build lại toàn bộ)
- Migration cơ sở dữ liệu chạy *trước* khi deploy service
- Cơ chế rollback: quay lại phiên bản trước trong < 5 phút
- Quản lý secret: không hardcode mật khẩu trong cấu hình CI

Câu hỏi phân tích:

1. **Thiết kế pipeline:** vẽ sơ đồ CI/CD pipeline với đầy đủ các stage, gồm: phát hiện service thay đổi, build song song, cổng kiểm thử (test gate), cổng phê duyệt, triển khai
2. **Phát hiện service thay đổi** (nếu gom các service vào một mono-repo): Git diff → xác định service nào thay đổi → chỉ build service đó. Thiết kế thuật toán. Trường hợp biên: thư viện dùng chung thay đổi → build lại *tất cả* service?
3. **Thứ tự migration cơ sở dữ liệu:** Core Service phụ thuộc vào các bảng dùng chung. Academic Service phụ thuộc vào các bảng của Core. Thứ tự migration có quan trọng không? Thiết kế đồ thị phụ thuộc (dependency graph) cho các migration
4. **Chiến lược rollback:** Dùng canary deployment cho production. Nếu instance canary có tỷ lệ lỗi > 5% → tự động rollback. Thiết kế cơ chế (health check → metrics → quyết định → rollback). Vẽ sơ đồ?
5. **GitOps vs push-based:** CI/CD truyền thống chủ động đẩy (push) bản triển khai; GitOps (ArgoCD) kéo (pull) trạng thái mong muốn từ Git. So sánh trong bối cảnh LMS — nên dùng cách nào?

Capstone: Phỏng vấn thiết kế hệ thống — thiết kế "Online Judge" `L3`

Đề bài (dạng phỏng vấn 45 phút):

Kịch bản

"Thiết kế hệ thống Online Judge (như LeetCode/HackerRank) cho 10.000 người dùng đồng thời. Hệ thống cho phép: người dùng nộp code (Python, Java, SQL), hệ thống chạy code trong sandbox, so sánh output, trả kết quả. Có chế độ thi (1.000 người dùng nộp bài cùng lúc) với bảng xếp hạng thời gian thực."

Gợi ý cấu trúc câu trả lời (theo khung phỏng vấn system design):

1. **Làm rõ yêu cầu** (5 phút): Yêu cầu chức năng? Phi chức năng (độ trễ, thông lượng, tính sẵn sàng)? Quy mô?
2. **Thiết kế tổng quan** (10 phút): Vẽ sơ đồ kiến trúc. Xác định các service, database, message queue
3. **Đi sâu** (20 phút): Chọn 2-3 thành phần quan trọng nhất:

    - **Thực thi trong sandbox**: Mỗi bài nộp một Docker container? Bảo mật? Giới hạn tài nguyên?
    - **Quản lý hàng đợi**: Chiến lược chia partition Kafka để lập lịch công bằng (chế độ thi)?
    - **Bảng xếp hạng**: CQRS + Redis sorted set? Tần suất cập nhật?
4. **Mở rộng và trade-off** (10 phút):
    - 1.000 bài nộp đồng thời → chiến lược mở rộng Judge Service?
    - Trade-off CAP cho bảng xếp hạng: thời gian thực (eventual consistency) hay chính xác (strong consistency)?
    - Ước tính chi phí: mỗi bài nộp = 1 Docker container × 10 giây = tốn bao nhiêu tài nguyên tính toán?

Tiêu chí đánh giá:

- ☐ Kiến trúc có ≥4 service rõ ràng
- ☐ Kiểu giao tiếp hợp lý (đồng bộ hay bất đồng bộ, khi nào dùng gì)
- ☐ Chiến lược quản lý dữ liệu (database-per-service, CQRS cho bảng xếp hạng)
- ☐ Các resilience pattern (circuit breaker cho Judge, DLT cho các bài nộp thất bại)
- ☐ Bảo mật (cô lập sandbox, xác thực bằng JWT)
- ☐ Phân tích khả năng mở rộng (mở rộng ngang Judge Service, chiến lược chia partition)
- ☐ Thảo luận trade-off (phân tích rõ ≥3 trade-off)

Ghi chú: Các bài tập case study dựa trên thông tin công khai từ engineering blog và tài liệu của các công ty (Netflix, Amazon, Shopify, Stripe). Các dữ kiện đã được đơn giản hóa cho mục đích giáo dục — nên tham khảo nguồn gốc để có bức tranh đầy đủ.